

Fibonacci Subsequence Explanation

873. Length of Longest Fibonacci Subsequence

Let's understand this problem from **zero** → **brute force** → **intuition** → **optimal DP** → **code** → **dry run**.

1. What is the problem asking?

You are given a **strictly increasing** array.

A subsequence is Fibonacci-like if:

$$x_i + x_{i+1} = x_{i+2}$$

For example:

```
[1, 2, 3, 5, 8]
```

because:

```
1 + 2 = 3
2 + 3 = 5
3 + 5 = 8
```

So its length is 5.

We need to find the **longest Fibonacci-like subsequence** that can be obtained from **arr**.

If no Fibonacci-like subsequence of length at least 3 exists, return 0.

2. Important observation

Suppose we have:

```
[1, 2, 3, 5, 8]
```

If we know the **last two numbers**:

```
1, 2
```

then the next number is completely determined:

```
1 + 2 = 3
```

Then:

```
2 + 3 = 5
```

Then:

```
3 + 5 = 8
```

So the key observation is:

If we know the last two elements of a Fibonacci-like subsequence, we know exactly what previous element is required.

For example, suppose our current pair is:

```
3, 5
```

To extend the sequence, we need:

```
3 + 5 = 8
```

But when working **backwards**, if the last two elements are:

```
5, 8
```

then the previous element must be:

```
8 - 5 = 3
```

This subtraction is the key to the DP solution.

3. Brute-force idea

One straightforward thought is:

Try every possible subsequence and check whether it is Fibonacci-like.

But there are 2^n subsequences.

With:

```
n <= 1000
```

that's completely impossible.

We need something much better.

4. Think about choosing two starting elements

A Fibonacci sequence is determined by its first two elements.

For example:

```
1, 2
```

determines:

```
1, 2, 3, 5, 8, ...
```

So we could try every pair:

```
(arr[0], arr[1])
(arr[0], arr[2])
(arr[0], arr[3])
...
```

There are approximately:

$$O(n^2)$$

pairs.

For each pair, we could keep generating the next Fibonacci number and search for it.

Since the array is sorted, we could use binary search.

This gives roughly:

$$O(n^2 \log n)$$

which might work for $n = 1000$, but we can do even better.

5. Can we avoid repeatedly searching?

Yes.

Since we repeatedly ask:

"Does this value exist in the array?"

we can build a hash map:

```
value → index
```

For example:

```
arr = [1,2,3,4,5,6,7,8]

map:
1 → 0
2 → 1
3 → 2
4 → 3
5 → 4
6 → 5
7 → 6
8 → 7
```

Then checking whether a value exists takes approximately:

```
0(1)
```

But we still need a way to remember sequences we've already calculated.

That's where **DP** comes in.

6. The DP state

This is the most important part.

Define:

```
C++
dp[i][j]
```

as:

The length of the longest Fibonacci-like subsequence whose **last two elements are arr[i] and arr[j]**.

So:

```
dp[i][j]
  ↓
... → arr[i] → arr[j]
```

For example:

```
arr = [1,2,3,5,8]
dp[2][3]
```

means:

```
... → 3 → 5
```

What could come before 3?

We need:

```
previous + 3 = 5
```

Therefore:

```
previous = 5 - 3 = 2
```

2 exists at index 1.

So:

```
1 → 2 → 3 → 5
```

If:

```
dp[1][2] = 3
```

then:

```
dp[2][3] = dp[1][2] + 1
          = 4
```

7. Why do we calculate arr[j] - arr[i]?

Suppose our current pair is:

```
arr[i], arr[j]
```

and we want to find the element before arr[i].

Fibonacci property says:

```
previous + arr[i] = arr[j]
```

Therefore:

```
previous = arr[j] - arr[i]
```

So we calculate:

```
C++
int prev = arr[j] - arr[i];
```

Then search for **prev**.

8. Why must the previous index be before i?

Remember this is a **subsequence**, so the order must remain unchanged.

Suppose:

```
k < i < j
```

Then:

```
arr[k] → arr[i] → arr[j]
```

is valid.

Therefore we need:

```
C++
k < i
```

Because the array is strictly increasing:

```
arr[k] < arr[i] < arr[j]
```

and therefore if:

```
prev = arr[j] - arr[i]
```

exists, its index will naturally be before **i** for a valid Fibonacci extension.

9. DP transition

Suppose:

```
arr[k] + arr[i] = arr[j]
```

Then:

```
arr[k] → arr[i] → arr[j]
```

can extend the sequence ending with:

```
arr[k] → arr[i]
```

Therefore:

```
C++
dp[i][j] = dp[k][i] + 1;
```

This is the heart of the solution.

10. What if the previous element doesn't exist?

Suppose:

```
arr = [1, 2, 4]
```

Consider:

```
2, 4
```

Required previous value:

```
4 - 2 = 2
```

But the previous value must come **before 2**, and there isn't another 2.

So we can't form a Fibonacci sequence of length 3.

We initialize every pair as:

```
C++
dp[i][j] = 2;
```

because any two elements form a sequence of length 2, but it isn't Fibonacci-like yet.

Only when we find a valid previous element do we extend it to length 3+.

11. Complete algorithm

Step 1

Create a map:

```
value → index
```

Step 2

Consider every pair:

```
i < j
```

Step 3

Calculate the required previous value:

```
C++
prev = arr[j] - arr[i];
```

Step 4

Check whether **prev** exists.

If yes, suppose:

```
index(prev) = k
```

Then:

```
C++
dp[i][j] = dp[k][i] + 1;
```

Otherwise:

```
C++
dp[i][j] = 2;
```

Step 5

Keep the maximum answer.

12. C++ Code

```
C++
class Solution {
public:
    int lenLongestFibSubseq(vector<int>& arr) {
        int n = arr.size();

        // value -> index
        unordered_map<int, int> mp;

        for (int i = 0; i < n; i++) {
            mp[arr[i]] = i;
        }

        // dp[i][j] = longest Fibonacci-like subsequence
        // ending with arr[i], arr[j]
```

```
vector<vector<int>> dp(n, vector<int>(n, 2));

int ans = 0;

for (int j = 0; j < n; j++) {
    for (int i = 0; i < j; i++) {
        // We need:
        // arr[k] + arr[i] = arr[j]
        // Therefore:
        // arr[k] = arr[j] - arr[i]

        int prev = arr[j] - arr[i];

        if (mp.count(prev)) {
            int k = mp[prev];

            // k must come before i
            if (k < i) {
                dp[i][j] = dp[k][i] + 1;
            }
        }

        ans = max(ans, dp[i][j]);
    }
}

// A Fibonacci-like sequence must have length >= 3
return ans >= 3 ? ans : 0;
};
```

13. Let's understand the DP visually

Imagine:

```
arr = [1, 2, 3, 5, 8]
      0 1 2 3 4
```

We eventually want:

```
1 → 2 → 3 → 5 → 8
```

The DP states are:

```
dp[0][1]
1,2
length = 2

dp[1][2]
2,3

previous = 3 - 2 = 1
index of 1 = 0

dp[1][2] = dp[0][1] + 1
          = 2 + 1
          = 3

sequence:
1 → 2 → 3
```

Next:

```
dp[2][3]
3,5

previous = 5 - 3
          = 2

index of 2 = 1

dp[2][3] = dp[1][2] + 1
          = 3 + 1
          = 4

sequence:
1 → 2 → 3 → 5
```

Next:

```
dp[3][4]
5,8

previous = 8 - 5
          = 3

index of 3 = 2

dp[3][4] = dp[2][3] + 1
          = 4 + 1
          = 5
```

So:

```
1 → 2 → 3 → 5 → 8
length = 5
```

14. Complete dry run on Example 1

Input:

```
arr = [1,2,3,4,5,6,7,8]
```

Map:

```
1 → 0
2 → 1
3 → 2
4 → 3
5 → 4
6 → 5
7 → 6
8 → 7
```

Initially every:

```
dp[i][j] = 2
```

because two numbers alone give us length 2.

$j = 1 \rightarrow \text{arr}[j] = 2$

$i = 0$

Pair:

```
1, 2
```

Required previous:

```
2 - 1 = 1
```

Index of 1 = 0.

But:

```
k = i = 0
```

We need $k < i$.

So we can't extend.

```
dp[0][1] = 2
```

$j = 2 \rightarrow \text{arr}[j] = 3$

$i = 0$

Pair:

```
1,3
```

Required:

```
3 - 1 = 2
```

Index of 2 = 1.

But:

```
1 > 0
```

so 2 occurs after 1.

Not valid.

```
dp[0][2] = 2
```

i = 1

Pair:

```
2,3
```

Required:

```
3 - 2 = 1
```

Index:

```
1 → 0
```

Now:

```
0 < 1
```

valid.

Therefore:

```
dp[1][2] = dp[0][1] + 1
          = 2 + 1
          = 3
```

Sequence:

```
1 → 2 → 3
```

j = 3 → arr[j] = 4

Consider pairs:

```
1,4
2,4
3,4
```

For 3,4:

```
4 - 3 = 1
```

But the sequence would need:

```
1 → 3 → 4
```

and:

```
1 + 3 != 4
```

Wait—our calculation says:

```
4 - 3 = 1
1 + 3 = 4
```

Actually it **is** valid.

So:

```
dp[2][3] = dp[0][2] + 1
          = 2 + 1
          = 3
```

Sequence:

```
1 → 3 → 4
```

This is another Fibonacci-like sequence.

j = 4 → arr[j] = 5

Important pair:

2,5

Required:

$$5 - 2 = 3$$

Index of 3 = 2.

So:

$$dp[1][4] = dp[2][1] + 1$$

But our DP is defined with smaller index first, so this particular direction doesn't form a valid state in that indexing arrangement.

The useful chain for the longest answer is:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 5$$

When we process:

3,5

we get:

$$5 - 3 = 2$$

Index of 2 = 1.

Therefore:

$$\begin{aligned} dp[2][4] &= dp[1][2] + 1 \\ &= 3 + 1 \\ &= 4 \end{aligned}$$

So:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 5$$

$j = 7 \rightarrow arr[j] = 8$

The important pair is:

5,8

Required previous:

$$8 - 5 = 3$$

Index of 3:

2

Therefore:

$$\begin{aligned} dp[4][7] &= dp[2][4] + 1 \\ &= 4 + 1 \\ &= 5 \end{aligned}$$

So we have:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 8$$

Therefore:

$$ans = 5$$

Final answer:

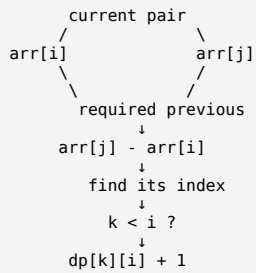
5

15. The most important mental model

Whenever you see:

Longest Fibonacci-like subsequence

think:



The whole DP is essentially:

```

previous    current    next
  ↓          ↓          ↓
  k          i          j

arr[k] + arr[i] = arr[j]

Therefore:
arr[k] = arr[j] - arr[i]

And:
dp[i][j] = dp[k][i] + 1

```

That's the core idea.

16. Why this is better than brute force

Brute force

Try every subsequence:

$$O(2^n)$$

Impossible.

Pair + binary search

Try every pair and search for the next element:

$$O(n^2 \log n)$$

Better, but unnecessary.

DP + HashMap

There are:

$$O(n^2)$$

pairs.

For each pair, hashmap lookup is approximately:

$$O(1)$$

Therefore:

$$O(n^2)$$

17. Complexity

For:

```
n = arr.size()
```

Time

We examine every pair:

```
(i, j), i < j
```

There are approximately:

$$\frac{n(n-1)}{2}$$

pairs.

Each pair performs an `unordered_map` lookup in average $O(1)$.

Therefore:

$$O(n^2)$$

Space

The DP table contains:

`n × n`

elements.

Therefore:

$$O(n^2)$$

The hashmap requires $O(n)$ additional space.

Overall:

$$O(n^2)$$

space.

One-line intuition to remember

For every pair (i, j) , ask: "What number must come before $arr[i]$ to make $arr[i] + arr[j]$ Fibonacci-like?" That number is $arr[j] - arr[i]$; if it exists at index k , extend the already-computed sequence $dp[k][i]$.

Exactly — your thinking is correct. We need to understand **why j starts from 0**, even though we need a pair $i < j$.

Look at the loops

```
C++
for (int j = 0; j < n; j++) {
    for (int i = 0; i < j; i++) {
```

The important part is:

```
C++
i < j
```

This automatically guarantees that we only consider **different elements**.

What happens when $j = 0$?

```
C++
j = 0
```

Then the inner loop becomes:

```
C++
for (int i = 0; i < 0; i++)
```

There is **no** i satisfying this condition.

So nothing happens.

Therefore, starting j from 0 is completely safe, but **not necessary**.

You could write:

```
C++
```

```
for (int j = 1; j < n; j++) {
    for (int i = 0; i < j; i++) {
        ...
    }
}
```

This is slightly cleaner because we know that $j = 0$ cannot make a pair.

Why can't the same element make a pair?

Suppose:

```
arr = [1, 2, 3, 4]
      0 1 2 3
```

We want pairs like:

```
(i, j)
(0, 1) → 1, 2
(0, 2) → 1, 3
(0, 3) → 1, 4
(1, 2) → 2, 3
(1, 3) → 2, 4
(2, 3) → 3, 4
```

But we **don't** want:

```
(0, 0) → 1, 1
(1, 1) → 2, 2
(2, 2) → 3, 3
```

because a pair should consist of **two different positions**.

That's why we use:

```
C++
i < j
```

Why not use $i \leq j$?

If we did:

```
C++
for (int i = 0; i <= j; i++)
```

then when:

```
i = j
```

we would get:

```
arr[j], arr[j]
```

For example:

```
i = 2
j = 2
arr[i] = 3
arr[j] = 3
```

We're treating the same element twice.

That's invalid for a subsequence.

So:

```
C++
i < j
```

is important.

Why can j start from 1?

Because j represents the **second element** of the pair.

The smallest possible pair is:

```
i = 0
j = 1
```

So we can simply write:

```
C++
for (int j = 1; j < n; j++) {
    for (int i = 0; i < j; i++) {
        int prev = arr[j] - arr[i];
        if (mp.count(prev)) {
            int k = mp[prev];
            if (k < i) {
                dp[i][j] = dp[k][i] + 1;
            }
        }
        ans = max(ans, dp[i][j]);
    }
}
```

This is **equivalent** to your original code.

Visualize the loop

For:

```
arr = [1,2,3,4,5]
      0 1 2 3 4
```

j = 1

```
      j
      ↓
[1, 2, 3, 4, 5]
  ↑
  i
pair = (0,1)
```

j = 2

```
      j
      ↓
[1, 2, 3, 4, 5]
  ↑ ↑
  i possible i's
pairs:
(0,2)
(1,2)
```

j = 3

```
      j
      ↓
[1, 2, 3, 4, 5]
  ↑ ↑ ↑
  i i i
pairs:
(0,3)
(1,3)
(2,3)
```

j = 4

```
      j
      ↓
[1, 2, 3, 4, 5]
  ↑ ↑ ↑ ↑
  i i i i
pairs:
(0,4)
(1,4)
(2,4)
(3,4)
```

So the total pairs are:

```
j = 1 → 1 pair
j = 2 → 2 pairs
```

```
j = 3 → 3 pairs
j = 4 → 4 pairs
```

Total:

$$1 + 2 + 3 + 4 = \frac{n(n-1)}{2}$$

which is exactly all possible pairs $i < j$.

So remember:

```
C++
for (int j = 1; j < n; j++)
```

is slightly cleaner, but:

```
C++
for (int j = 0; j < n; j++)
```

is also **correct**, because when $j = 0$, the inner loop simply executes zero times.

And yes — $i < j$ is **what prevents the same element/position from being used twice and also maintains subsequence order**.

ChatGPT can make mistakes. Check important info.